

HOG + LBP + SVM ile Göz Bölgesi Üzerinden Deepfake Tespiti

Kod ve Deney Açıklama Raporu

Combined Eye ROI tabanlı klasik makine öğrenmesi notebook'unun veri güvenliği, özellik çıkarımı, SVM seçimi, kalite kapıları, test değerlendirme ve çıktı üretim sürecinin teknik açıklaması

Alan	Değer
Deney yaklaşımı	HOG + LBP özellik füzyonu + SVM
İncelenen bölge	Göz - Combined Eye ROI
Önceden eğitilmiş model	Kullanılmıyor
Pozitif sınıf	Fake
Seed	42
Gerçek Run ID	20260807_1119_eye_hog_lbp_svm_seed42
Nihai seçilen sınıflandırıcı	RBF SVM

1. Raporun Amacı

Bu rapor, HOG + LBP + SVM tabanlı göz bölgesi deepfake tespit notebook'unun ne yaptığını, neden bu şekilde tasarlandığını ve her ana kod bloğunun deney açısından hangi görevi üstlendiğini açıklamak amacıyla hazırlanmıştır. Notebook derin öğrenme ağı eğitmek yerine klasik makine öğrenmesi yaklaşımı kullanır. Combined Eye ROI görüntülerinden önce elle tasarlanmış görsel özellikler çıkarılır; ardından bu özellikler SVM sınıflandırıcısına verilir.

Temel bilimsel soru şudur: Göz bölgesindeki kenar/yönelim bilgisi ile mikro-doku bilgisinin birlikte kullanılması, Real ve Fake örnekleri ayırmada ne kadar faydalıdır?

2. Genel Çalışma Akışı

Adım	İşlem
1	Drive bağlantısı ve merkezi YAML config
2	Eye ROI metadata şema doğrulaması
3	Veri muhasebesi ve leakage quality gate'leri
4	Combined-eye görüntülerini grayscale okuyup 128x64 resize
5	HOG ve LBP özellik çıkarımı
6	HOG + LBP concatenation ile özellik füzyonu
7	StandardScaler'ın yalnız train setinde fit edilmesi
8	12 SVM adayının validation setinde karşılaştırılması
9	Validation F1 ile model + threshold seçimi
10	Atomik checkpoint ve fresh-load inference testleri
11	Test setinde frame-level ve metadata-group-level değerlendirme
12	Metrik, tahmin, grafik, config, environment ve manifest çıktıları

2.1. Neden HOG + LBP + SVM?

HOG, göz çevresindeki kenarların, göz kapağı sınırlarının, kirpiklerin ve lokal şekil değişimlerinin yönelim bilgisini özetler.

LBP, iris çevresi, göz çevresi deri dokusu ve mikro-kontrast örüntülerini yerel komşuluk ilişkileri üzerinden temsil eder.

İki özellik uç uca birleştirilerek aynı örnekte hem geometrik/yönelim hem de mikro-doku bilgisi kullanılır.

SVM, bu yüksek boyutlu özellik uzayında Real ve Fake sınıflarını ayıran karar sınırını öğrenir. Yaklaşım hafif, açıklanabilir ve deep learning modellerine karşı kullanılabilecek klasik bir baseline'dır.

3. Konfigürasyon ve Deney Ayarları

Parametre	Notebook değeri	Amaç
Görüntü boyutu	128 x 64	HOG/LBP için sabit geometri
HOG orientations	9	Dokuz gradyan yönünü temsil etmek
HOG pixels_per_cell	8 x 8	Yerel yönelim histogramları
HOG cells_per_block	2 x 2	Komşu hücreleri birlikte normalize etmek
HOG block norm	L2-Hys	Kontrast/aydınlık değişimine dayanıklılık
LBP points	8	Sekiz komşuluk ilişkisi
LBP radius	1	Yerel mikro-doku
LBP method	uniform	Düşük boyutlu ve kararlı histogram
SVM kernels	linear, rbf	Doğrusal ve doğrusal olmayan sınırları karşılaştırmak
C adayları	0.1, 1.0, 10.0	Düzenileştirme/uyum dengesini taramak
RBF gamma	scale, 0.001, 0.0001	RBF etki alanını taramak
Class weight	balanced	Sınıf sayı farkını otomatik dengelemek
Seçim metriği	Validation F1	Precision ve Recall'u birlikte optimize etmek

Figure DPI	150	Rapor görsellerinde yeterli çözünürlük
------------	-----	--

3.1. YAML hata koruması

Config dosyası bozuk YAML üretirse notebook belirsiz biçimde devam etmez. Bozuk dosya .yaml.broken uzantısıyla yedeklenir, varsayılan geçerli config yeniden yazılır ve tekrar okunur. Ayrıca gerekli config anahtarlarının tamamının bulunduğundan doğrulanır.

4. Run Klasörü, Tekrar Üretilirlik ve Dosya Güvenliği

Her çalışma tarih-saat, bölge, yöntem ve seed içeren benzersiz bir Run ID ile Deney 1/Sonuçlar altında ayrı klasöre yazılır. Mevcut run klasörünün üzerine bilinçsizce yazılması engellenir; resume_run_id yalnızca bilinçli devam senaryosu için kullanılabilir.

Öge	Yol / davranış
Girdi	Deney 1/Göz/eye_roi_output
Metadata	Göz/eye_roi_output/metadata.csv
Çıktı	Deney 1/Sonuçlar/<run_id>/
Kaynak veri	Salt okunur; taşınmaz/silinmez/yeniden adlandırılmaz
Seed	Python + NumPy + PYTHONHASHSEED = 42
Log	logs/run.log

JSON, CSV, NPZ ve joblib çıktıları geçici .tmp dosyalarına yazılır, tekrar okunarak doğrulanır ve yalnızca başarılı doğrulamadan sonra os.replace ile atomik biçimde hedef dosyaya geçirilir.

5. Eye ROI Metadata Doğrulaması ve Veri Muhasebesi

Notebook doğrudan göz ROI metadata'sını kullanır ve sample_id, source_frame, label, split, video_id, frame_stem, face_id, combined_eye_path ve status gibi zorunlu alanların varlığını kontrol eder.

Önemli düzeltme: no_face satırlarında sample_id boş olabilir. Bu nedenle SUCCESS satırlarında gerçek sample_id zorunlu ve benzersiz tutulurken, SKIPPED/ERROR satırlarına yalnız izlenebilirlik amacıyla deterministik audit ID üretilir. Bu audit ID gerçek ROI kimliği gibi değerlendirilmez.

Gerçek run veri muhasebesi	Değer
Toplam metadata satırı	3,097
SUCCESS	2,986
SKIPPED (no_face)	111
ERROR	0
Eğitime uygun combined-eye örneği	2,986
Üretilen audit ID	111

Split	REAL	FAKE	Toplam
Train	1,197	1,191	2,388
Validation	155	141	296
Test	146	156	302

5.1. SHA256 ve exact-content leakage kontrolü

SUCCESS statüsündeki 2,986 combined-eye dosyasının SHA256 değeri gerçek dosya içeriğinden hesaplanır. Aynı görüntü içeriği farklı splitlerde bulunursa deney durdurulur. Gerçek çalıştırmada cross-split duplicate SHA256 sayısı 0'dır.

Aynı dosya yolunun farklı splitlerde görünmesi de ayrıca kontrol edilir; gerçek run'da bu sayı da 0'dır.

5.2. video_id alanının bilimsel yorumu

Notebook mevcut metadata'daki video_id alanını olduğu gibi korur; ancak bu alan real_train, fake_test gibi split/sınıf grup kimliği taşıyabildiğinden authoritative original source-video kimliği olarak kabul edilmez.

Bu nedenle mevcut video_id grupları arasında train/val/test kesişimi 0 olsa da gerçek video-level leakage durumu PASS olarak işaretlenmez. Kalite kapısında durum NOT_VERIFIABLE_FROM_CURRENT_METADATA olarak raporlanır.

6. Görüntü Ön İşleme

Her combined-eye ROI OpenCV ile grayscale okunur ve 128x64 piksele yeniden boyutlandırılır. İşlem RAM üzerindeki kopyada yapılır; Drive'daki kaynak ROI dosyasına yazılmaz.

İşlem	Neden kullanılıyor?
Grayscale	Renk bilgisini kaldırıp yapı/dokuya odaklanmak
128x64 resize	Tüm örnekleri sabit HOG/LBP geometrisine getirmek
uint8 kontrolü	Piksel veri tipini tutarlı tutmak
INTER_AREA	Küçültmede kararlı yeniden örnekleme

7. HOG Özellik Çıkarımı

HOG (Histogram of Oriented Gradients), görüntüdeki lokal gradyan yönelimlerini histogramlar. Göz bölgesinde göz kapağı kıvrımı, kirpik çizgileri, göz köşeleri ve genel geometrik sınırlar gibi şekil/yönelim ipuçlarını temsil etmek için kullanılır.

Notebook 9 yönelim, 8x8 hücre, 2x2 blok ve L2-Hys normalizasyonu kullanır.

Gerçek çalıştırmada her görüntü için HOG özellik boyutu 3,780'dir.

8. LBP Özellik Çıkarımı

LBP (Local Binary Pattern), merkez piksel ile komşu pikseller arasındaki yerel yoğunluk ilişkilerini kodlayan klasik bir mikro-doku tanımlayıcısıdır. Göz çevresi deri dokusu, iris çevresi ve ince kontrast değişimlerinin Real/Fake ayırımına katkısını ölçmek için kullanılır.

P=8, R=1 ve uniform yöntem kullanılır. LBP görüntüsü doğrudan modele verilmez; normalize histogram üretilir.

Gerçek çalıştırmada LBP histogram boyutu 10'dur.

9. HOG + LBP Özellik Füzyonu

Özellik	Boyut
HOG	3,780
LBP	10
Birleştirilmiş vektör	3,790

İki vektör np.concatenate ile uç uca eklenir. Böylece SVM aynı örneğin hem şekil/yönelim hem de mikro-doku bilgisini birlikte görür. Birleştirilmiş vektörde NaN veya Inf tespit edilirse süreç hata verir.

10. Classical-ML Smoke Test

Tüm veri üzerinde uzun işlem başlamadan önce train setinden iki Real ve iki Fake örnek seçilir. Bu dört örnekte HOG + LBP çıkarımı, StandardScaler fit'i, Linear SVM eğitimi ve tahmin çalıştırılır.

Bu test feature extraction, scaler ve SVM zincirinin temel olarak çalıştığını doğrular. Gerçek run'da Classical-ML smoke test PASS olmuştur ve özellik boyutu 3,790 olarak doğrulanmıştır.

11. Feature Cache Mekanizması

Train, validation ve test özellikleri ayrı NPZ cache dosyalarına atomik olarak kaydedilir. Cache dosyası varsa yeniden HOG/LBP hesaplamak yerine doğrudan yüklenir.

Split	Gerçek extraction süresi
Train - 2,388 örnek	34.68 s
Validation - 296 örnek	3.37 s
Test - 302 örnek	3.13 s

Cache yaklaşımı özellikle Colab kesintileri veya aynı run'ın devam ettirilmesi durumunda ciddi süre kazandırır.

12. StandardScaler: Neden Yalnız Train Üzerinde?

HOG ve LBP özelliklerinin sayısal ölçekleri farklıdır. SVM özellikle RBF kernel kullanırken feature ölçeğine duyarlıdır; bu nedenle özellikler StandardScaler ile standartlaştırılır.

Scaler yalnız X_train üzerinde fit edilir. Validation yalnız transform edilir; test ise model ve threshold seçimi tamamlanana kadar dönüştürülmez. Bu tasarım validation/test istatistiklerinin train aşamasına sızmasını engeller.

13. SVM Adayları ve Model Seçimi

Notebook tek bir SVM seçip test etmez. Linear ve RBF kernel için önceden tanımlı hiperparametre kombinasyonlarını train setinde fit eder ve yalnız validation setinde karşılaştırır.

Kernel	C	Gamma	Aday sayısı
Linear	0.1, 1.0, 10.0	Yok	3
RBF	0.1, 1.0, 10.0	scale, 0.001, 0.0001	9
Toplam	-	-	12

Her aday için decision_function skoru üretilir. Precision-Recall eğrisi üzerinden validation F1'i maksimum yapan karar threshold'u aday bazında ayrıca seçilir.

13.1. Gerçek run'da seçilen model

Seçilen ayar	Değer
Kernel	RBF
C	10.0
Gamma	0.0001
Threshold	-0.543336
Selection split	Validation
Selection metric	F1
Validation Accuracy	0.6284
Validation Precision	0.5721
Validation Recall	0.8723

Validation F1	0.6910
Validation Specificity	0.4065
Validation ROC-AUC	0.6794
Validation Avg. Precision	0.6485

12 adayın tamamının taranması yaklaşık 557 saniye sürmüştür. En iyi validation F1, RBF kernel C=10 ve gamma=0.0001 kombinasyonunda elde edilmiştir.

14. Threshold Neden Ayırı Seçiliyor?

SVC decision score üretir ve varsayılan karar sınırı 0 olabilir. Notebook sabit 0 kullanmak yerine validation setindeki Precision-Recall eğrisini tarayarak F1'i en yüksek yapan threshold'u seçer.

Seçilen threshold daha sonra sabitlenir ve test setinde değiştirilmez. Böylece test sonuçlarına bakarak karar sınırı ayarlama yoluyla veri sızıntısı oluşması engellenir.

15. Checkpoint ve Yeniden Yükleme Kalite Kapıları

Bu deney PyTorch sinir ağı değildir; dolayısıyla optimizer_state_dict, scheduler veya epoch tabanlı gradient training state'i doğal olarak yoktur. Notebook bunları varmış gibi üretmek yerine N/A olarak açıkça işaretler.

Çıktı	İçerik / amaç
best.ckpt	Seçilen SVC + scaler + threshold + config + RNG state
last.ckpt	Son klasik-ML kayıt durumu
checkpoint_quality_gate.ckpt	Atomik save/reload doğrulaması
svm_model.joblib	Bağımsız inference için model paketi

Checkpoint yeniden yüklendikten sonra validation decision score'larının orijinal modelde np.allclose düzeyinde aynı olduğu doğrulanır. Ayrıca svm_model.joblib sıfırdan yüklenip örnek inference üreterek fresh-load testinden geçirilir.

16. Quality Gate Sonuçları

Kontrol	Durum
Metadata schema	PASS
Current video_id group split test	PASS_FOR_CURRENT_VIDEO_ID_GROUPS
True video-level split test	NOT_VERIFIABLE_FROM_CURRENT_METADATA
Data accounting	PASS
Exact duplicate SHA256	PASS
Numerical feature check	PASS
Classical ML smoke test	PASS
PyTorch forward/backward	NOT_APPLICABLE_CLASSICAL_ML
Checkpoint atomic save/reload	PASS
Fresh-load inference	PASS
Test used for selection?	False

True video-level testinin NOT_VERIFIABLE olması hata değildir; mevcut metadata'nın original source-video kimliğini kanıtlamadığını açık biçimde raporlar. Bu, yanlış bir PASS iddiası üretmekten daha doğru ve denetlenebilir bir davranıştır.

17. Nihai Test Değerlendirmesi

Test setine model ve threshold seçimi tamamlandıktan sonra ilk kez bakılır. Frame düzeyinde her combined-eye ROI bağımsız örnek olarak değerlendirilir.

Frame-level test metriği	Değer
Accuracy	0.5927
Precision	0.5714
Recall	0.8462
F1	0.6822
Specificity	0.3219
ROC-AUC	0.6032
Average Precision	0.6051

Frame-level sonuçta Fake recall yüksektir (%84.62); buna karşılık specificity %32.19'dur. Bu nedenle model Fake örnekleri yakalamaya eğilimli, fakat Real örneklerde yanlış alarm üretmeye açıktır. ROC-AUC 0.6032 olması, ayırtırma gücünün rastgele seviyenin üzerinde fakat sınırlı olduğunu gösterir.

17.1. Group-level (metadata video_id) sonuçları

Group-level metrik	Değer
Accuracy	0.5000
Precision	0.5000
Recall	1.0000
F1	0.6667
Specificity	0.0000
ROC-AUC	1.0000
Average Precision	1.0000

Bu değerler gerçek video-level genelleme metriği olarak yorumlanmamalıdır. Mevcut metadata'da train, validation ve test için yalnızca 2'şer video_id grubu vardır ve bu video_id alanı gerçek orijinal video kimliği olmayabilir.

ROC-AUC=1.0 görünmesine rağmen accuracy=0.5 ve specificity=0 olması, score sıralamasının iki grup arasında doğru olabilese bile seçilen threshold'un tüm group kararlarını Fake tarafına taşıdığını gösterir. Küçük grup sayısı nedeniyle bu sonuç keşifsel/yardımcıdır.

18. Üretilen Grafikler

Grafik	Amaç
dataset_distribution.png	Split-sınıf dağılımı
svm_validation_f1_comparison.png	12 SVM adayının validation F1 karşılaştırması
test_confusion_matrix_frame_level.png	Frame-level hata dağılımı
test_confusion_matrix_group_level.png	Metadata-group-level hata dağılımı
test_roc_curve_frame_level.png	Frame-level ROC
test_roc_curve_group_level.png	Group-level ROC
test_precision_recall_curve_frame_level.png	Frame-level PR
test_precision_recall_curve_group_level.png	Group-level PR
test_decision_score_distribution.png	Real/Fake decision score dağılımı ve threshold

Tüm grafiklerin kısa kenarı en az 600 piksel olmalıdır. Gerçek run'da 9 grafiğin tamamı quality_pass=True çıkmıştır.

19. Sonuç Klasörüne Kaydedilen Yapı

Her run aşağıdaki mantıkla saklanır:

```
<run_id>/
checkpoints/
  best.ckpt
  last.ckpt
  checkpoint_quality_gate.ckpt
logs/
  run.log
metrics/
  data_accounting.json
  svm_validation_search.csv
  svm_selection.json
  quality_gates.json
  test_metrics_frame_level.json
  test_metrics_group_level.json
  test_metrics_summary.csv
  fresh_load_inference_test.json
  figure_quality_audit.csv
predictions/
  test_predictions_frame_level.csv
  test_predictions_group_level.csv
figures/
artifacts/
  metadata_used.csv
  features_train.npz
  features_val.npz
  features_test.npz
  feature_dimensions.json
  feature_pipeline.json
  feature_preprocessing.joblib
  svm_model.joblib
config_resolved.yaml
requirements_lock.txt
environment.json
output_manifest.csv
run_summary.json
```

20. Notebook'un Yapmadığı Şeyler

- Combined-eye görüntülerini yeniden kırpamaz veya kaynak ROI'lerin üzerine yazmaz.
- Frame düzeyinde yeni rastgele train/validation/test bölmesi oluşturmaz; metadata splitlerini kullanır.
- Pretrained CNN, ResNet, EfficientNet, Xception veya Transformer kullanmaz.
- HOG veya LBP'yi eğitmez; bunlar deterministik feature extractor'lardır.
- Test setini SVM veya threshold seçimi için kullanmaz.

- Metadata video_id alanını gerçek source-video kimliği diye uydurmaz.
- Klasik SVM deneyine PyTorch optimizer/epoch state'i yapay biçimde eklemes.

21. Genel Teknik Değerlendirme

Bu notebook göz bölgesine özgü, izlenebilir ve yeniden üretilebilir bir klasik makine öğrenmesi baseline'ı oluşturur. Combined-eye ROI'den HOG ile yönelim/geometri, LBP ile mikro-doku özellikleri çıkarılır; 3,790 boyutlu birleşik temsil train-fitted StandardScaler ile normalize edilir ve validation setinde 12 SVM adayı arasından en iyisi seçilir.

Gerçek çalıştırmada RBF SVM ($C=10$, $\gamma=0.0001$) seçilmiş ve validation $F1=0.691$ elde edilmiştir. Testte frame-level recall=0.846 ve $F1=0.682$ iken ROC-AUC=0.603 ve specificity=0.322'dir. Bu sonuç, HOG+LBP özelliklerinin Fake örnekleri yakalama açısından yararlı sinyal taşıdığını; ancak Real/Fake ayrımında tek başına güçlü bir çözüm olmadığını göstermektedir.

En önemli metodolojik güçlü yönler; test izolasyonu, train-only scaling, validation-only threshold seçimi, exact-content SHA256 leakage kontrolü, atomik kayıt, fresh-load inference ve metadata anlamı hakkında yanlış iddiadan kaçınılmasıdır.